



LastPrice

A Frictionless Silent Auction & Physical Escrow Marketplace

Course Title: Software Engineering and System Analysis Lab

Course Code: CSE 356 / CSE 0613226

Semester: Spring 2026

**A Project Report Submitted in Partial Fulfillment of the Requirements for the
Degree of Bachelor of Science in Computer Science and Engineering**

Submitted By:

Mehedi Hassan Bhuiyan	ID: 0432320005101080
Md. Masud Rahman	ID: 0432320005101064
Hasib Al Mahmud Siddique	ID: 0432320005101095
Rudro Antony Mrong	ID: 0432320005101059

Supervised By:

Dhrubo Barua
Md. Faysal

Faculty Members, Department of CSE
University of Information Technology and Sciences

Date of Submission: 20th May 2026

Abstract

Peer-to-peer (P2P) online commerce frameworks face major challenges: high bargaining friction, information asymmetry, and personal safety risks during offline cash-on-delivery transactions. Traditional e-commerce sites rely on drawn-out text negotiations, while online open English auctions trigger continuous bidding anxiety and invite shilling and reserve-sniffing exploits.

To solve these issues, this report presents **LastPrice**, a novel web-based silent auction marketplace integrated with a physical escrow validation protocol. LastPrice eliminates open, sequential bidding by enforcing a strict, double-blind three-bid silent auction system. Buyers are given exactly three private bids to state their true valuation on active listings. The system protects the seller's secret reserve floor price by translating bidder proximity into broad visual indicators (Red, Yellow, Green) rather than exposing numerical margins. Auctions are resolved automatically by a microsecond-accurate cron engine that resolves ties in favor of the first-mover.

Physical security during face-to-face handovers is maintained through a two-factor physical handshake verification protocol. Upon auction resolution, the system issues distinct, cryptographically linked 6-digit keys to both parties. The funds are held in state until both keys are entered and verified inside the escrow portal. Built on a modular PostgreSQL database, Express/Node.js, and styled with a cinematic high-fidelity dark Sepia design system, LastPrice offers a secure, fast, and frictionless paradigm for modern peer-to-peer commerce.

Contents

1	Introduction	5
1.1	Project Background	5
1.2	Problem Statement	5
1.3	Research Motivation	5
1.4	Goals and Objectives	6
1.5	Scope and Limitations	6
1.6	Report Organization	6
2	Methodology	7
2.1	Game-Theoretic Foundations of the 3-Bid Silent Auction	7
2.2	Vaulted Price Tension Shielding Model	7
2.3	Chronological First-Mover Tie Breaker Criteria	7
2.4	Cryptographic Physical Handshake Escrow Protocol	8
2.5	Latency-Compensated Time Synchronization	8
3	Software Requirement Specification	9
3.1	User Personas and Roles	9
3.2	Functional Requirements	9
3.3	Non-Functional Requirements	10
3.3.1	Performance and Scalability	10
3.3.2	Security and Encryption	10
3.3.3	Reliability and Fault Tolerance	10
3.3.4	Usability and Aesthetics	10
4	System Design	11
4.1	Use Case Diagram	11
4.2	Level-1 Data Flow Diagram (DFD)	12
4.3	Entity Relationship Diagram (ERD)	13
4.4	Class Diagram	14
4.5	Interaction Sequence Diagram	16
4.6	State Machine Diagram	16
5	System Implementation	18
5.1	Database Schema Definition	18
5.2	Chronological First-Mover Tie Breaker Core	19
5.3	Strict 3-Bid Capping Enforcement Middleware	19
5.4	Price Tension Shielding Logic	19
5.5	Real-Time Client-Server Clock Synchronization	20
6	System Walkthrough & User Interfaces	21
6.1	Step 1: Landing Page & Portal Discovery	21
6.2	Step 2: Secure User Authentication	22
6.3	Step 3: Interactive Marketplace Listings	22
6.4	Step 4: Seller Listing Creation	23
6.5	Step 5: Bidding Arena & Tension Feedback	25
6.6	Step 6: Centralized User Dashboard	25
6.7	Step 7: Seller Listing Console	26

6.8	Step 8: Escrow Mutual Verification Portal	27
7	System Testing & Evaluation	29
7.1	Test Case Scenarios	29
7.2	Security Audits and Attack Vector Mitigations	29
7.3	Database Indexing & Query Optimization	30
8	Results and Discussion	31
8.1	Visual Proximity Shielding Analysis	31
8.2	Escrow Handshake Verification Integrity	31
8.3	Comparative Platform Analysis	31
9	Challenges and Future Scope	32
9.1	Key Development Challenges	32
9.2	Future Technical Scope	32
10	Conclusion	34

List of Figures

4.1	LastPrice Unified Use Case Diagram	12
4.2	LastPrice Level-1 Data Flow Diagram	13
4.3	LastPrice Entity Relationship Diagram	13
4.4	LastPrice Object-Oriented Class Diagram	15
4.5	Interactive Bidding and Verification Sequence Diagram	16
4.6	Listing Lifecycle State Machine Diagram	17
6.1	LastPrice Landing Homepage Interface	21
6.2	User Authentication Portal	22
6.3	Active Marketplace Listings Grid	23
6.4	Create Listing Interface for Sellers	24
6.5	Bidding Arena showing Real-time Tension and 3-Bid Cap	25
6.6	Unified User Dashboard Hub	26
6.7	Seller Listing Management Panel	27
6.8	Double-Verification Handshake Escrow Page	28

List of Tables

3.1	Extended System Functional Requirements Specifications	9
7.1	System Evaluation Test Cases	29
7.2	Database Index Optimization Impact	30
8.1	Comparative C2C E-Commerce Platform Matrix	31

Chapter 1

Introduction

1.1 Project Background

The rise of consumer-to-consumer (C2C) e-commerce platforms has changed how people trade pre-owned goods. However, modern peer-to-peer marketplaces are still plagued by negotiation friction. Current e-commerce platforms are divided into two main categories: manual negotiation listings and open auction boards.

In manual negotiation frameworks (e.g., Bikroy, Facebook Marketplace, Craigslist), trading is slow. Buyers and sellers exchange endless messages trying to settle on a price. This manual process causes bargaining fatigue and high transaction drop-out rates. Conversely, open English auction systems (e.g., eBay) present alternative challenges: they trigger bidding anxiety, require constant countdown monitoring, and make platforms vulnerable to artificial price manipulation (shill bidding) and reserve-sniffing exploits.

LastPrice addresses these core inefficiencies. It moves away from open haggling and continuous bidding, introducing a specialized double-blind three-bid silent auction capping system. Buyers bid in private, completely hidden from other participants. Bidders are allowed exactly three bids to reach the seller's reserve floor price. Once the auction concludes, the system's background engine automatically processes the outcome, applying a first-mover advantage to resolve ties. Offline physical handovers are protected by a secure 6-digit mutual-code handshake verification system, protecting both users during face-to-face transactions.

1.2 Problem Statement

Contemporary peer-to-peer digital marketplaces suffer from several key vulnerabilities:

1. **Cognitive Haggling Fatigue:** Buyers and sellers waste time on manual negotiations. Sellers rarely get true market value, and buyers face constant communication friction.
2. **Exploitative Reserve Sniffing:** In open bidding platforms, dishonest buyers submit tiny, incremental bids to find the seller's minimum reserve price without ever intending to place a fair high-value bid.
3. **Handover Friction and Fraud:** Offline cash-on-delivery transactions rely entirely on verbal agreements. Sellers face payment disputes, and buyers are vulnerable to false claims of non-delivery.
4. **Client-Server Clock Drift:** Real-time bidding applications experience synchronization failures due to differences in client machine times. This clock drift leads to disputes over bids submitted in the closing seconds of an auction.

1.3 Research Motivation

Our main motivation is to build a highly efficient, automated system that eliminates transaction friction and ensures user safety. By limiting buyers to exactly three anonymous bids, the platform encourages bidders to state their true willingness-to-pay early. This design prevents bidding wars and stops reserve-sniffing bots. Additionally, adding a cryptographic physical escrow handshake ensures that face-to-face meetups are completely secure and verifiable.

1.4 Goals and Objectives

The core goals and software engineering objectives of LastPrice are:

- **Minimize Negotiation Overhead:** Replace continuous haggling with a three-bid cap, ensuring quick bid-to-listing matching.
- **Protect Reserve Floor Privacy:** Hide the seller's reserve floor behind a color-coded tension bar (Red, Yellow, Green), preventing bidders from reverse-engineering the exact minimum price.
- **Automate Bidding Arena Resolution:** Build a database-backed cron resolution engine that handles tie-breakers fairly using chronological priority.
- **Secure Face-to-Face Handovers:** Provide a two-factor physical validation portal requiring both parties to input matching keys to close a transaction.
- **Synchronize Real-Time Actions:** Implement a latency-compensated time synchronization API to maintain exact countdown consistency across all user terminals.

1.5 Scope and Limitations

LastPrice is designed as a localized Web 2.0 consumer-to-consumer trading application. The system focuses on automated price matching, tension feedback, and physical handshake escrows. However, this version does not handle digital online payment gateways, credit card processing, or shipment courier tracking. Instead, it is optimized for secure local meetups and cash-on-delivery transactions.

1.6 Report Organization

The remainder of this project report is structured as follows:

- **Chapter 2 (Methodology):** Outlines the game-theoretic foundations, price tension indicators, chronological tie-breaking, and time-sync mathematical modeling.
- **Chapter 3 (Software Requirement Specification):** Specifies user personas, comprehensive functional requirements, and detailed non-functional requirements.
- **Chapter 4 (System Design):** Illustrates the structural and behavioral design of LastPrice through unified UML diagrams, including Use Cases, DFDs, ERDs, Class Diagrams, Sequence Diagrams, and State Machines.
- **Chapter 5 (System Implementation):** Discusses database schemas, core controller code, middleware bid validation, and latency compensation frontend logic.
- **Chapter 6 (User Interfaces and Walkthrough):** Step-by-step documentation of the user interface flow, describing each of the 8 captured walkthrough screenshots.
- **Chapter 7 (System Testing and Evaluation):** Details the evaluation test matrix, security audit findings, and database indexing benchmarks.
- **Chapter 8 (Results and Discussion):** Discusses system efficiency, bidding behavior distributions, and a comparative analysis against existing platforms.
- **Chapter 9 (Challenges and Future Scope):** Summarizes implementation obstacles and outlines future development pathways.
- **Chapter 10 (Conclusion):** Summarizes the core outcomes of the academic project.

Chapter 2

Methodology

This chapter details the theoretical and mathematical modeling of LastPrice’s core mechanisms: the double-blind three-bid limit, the qualitative tension calculation, chronological first-mover priority, two-factor physical handshake verification, and drift-compensated client clock synchronization.

2.1 Game-Theoretic Foundations of the 3-Bid Silent Auction

Unlike traditional open ascending auctions (English auctions) where information is publicly revealed sequentially, LastPrice uses a double-blind silent auction model. Under this design, the utility U_i for bidder i with true valuation v_i who submits a bid $b_{i,r}$ in round $r \in \{1, 2, 3\}$ is defined as:

$$U_i(b_{i,r}) = \begin{cases} v_i - b_{i,r} & \text{if } b_{i,r} \text{ is the winning bid and } b_{i,r} \geq R \\ 0 & \text{otherwise} \end{cases} \quad (2.1)$$

where R represents the seller’s secret reserve floor price. In an open auction, bidders have an incentive to bid marginally above the current highest bid, leading to long bid sequences and shilling.

By limiting each buyer to exactly three bids, the game-theoretic strategy shifts. Bidders cannot slowly probe the seller’s threshold. To maximize their probability of winning, they must submit bids that represent their actual willingness-to-pay before their three attempts run out. This mechanism enforces rapid transaction resolution and eliminates the need for drawn-out negotiations.

2.2 Vaulted Price Tension Shielding Model

To protect the seller’s reserve floor R from sniffing scripts, the system calculates price proximity server-side. Rather than exposing numerical values, the system translates the bid b into a qualitative visual tension indicator $\tau(b)$:

$$\tau(b) = \begin{cases} \text{Green} & \text{if } b \geq R \\ \text{Yellow} & \text{if } 0.70R \leq b < R \\ \text{Red} & \text{if } b < 0.70R \end{cases} \quad (2.2)$$

The visual feedback returned to the client browser is restricted to these colored categories. Consequently, a bidder cannot determine the exact value of R through binary search bidding. They only learn whether their bid is far below, close to, or above the reserve floor, protecting the seller’s price boundary while maintaining active bidding engagement.

2.3 Chronological First-Mover Tie Breaker Criteria

If multiple buyers submit identical winning bid amounts, a fair tie-breaker is required. LastPrice implements a first-mover advantage. The winning bid is selected using chronological priority:

$$W(L) = \left\{ b_{i,r} \mid b_{i,r} \geq R \wedge b_{i,r} = \max_{j,s} b_{j,s} \wedge t(b_{i,r}) = \min [t(b_{j,s}) \mid b_{j,s} = b_{i,r}] \right\} \quad (2.3)$$

where $t(b_{j,s})$ is the microsecond-accurate server timestamp when bid $b_{j,s}$ was written to the PostgreSQL database. This ensures that the bidder who registered their valuation first is awarded the listing, incentivizing early bidding and rewarding active marketplace participation.

2.4 Cryptographic Physical Handshake Escrow Protocol

To secure face-to-face handovers, LastPrice implements a dual-key validation handshake. When an auction resolves successfully:

1. The system generates two distinct, secure 6-digit alphanumeric codes: K_B for the buyer and K_S for the seller.
2. These codes are cryptographically linked in the database to a pending transaction record.
3. During the physical meeting, the seller inspects the physical cash (or item), and the buyer inspects the product.
4. Once satisfied, they exchange their keys. Both K_B and K_S must be entered into the escrow portal.
5. The transaction executes successfully only when both matching keys are verified:

$$f(K_B, K_S) = \begin{cases} \text{Release Escrow / Complete} & \text{if } \text{Verify}(K_B) \wedge \text{Verify}(K_S) \\ \text{Maintain Pending State} & \text{otherwise} \end{cases} \quad (2.4)$$

This double-verification loop prevents one-sided claims of non-delivery or payment failure, making offline handovers fully secure.

2.5 Latency-Compensated Time Synchronization

To ensure client-side count downs are accurate, the frontend computes network latency and calculates the system clock drift offset θ :

$$\text{RTT} = T_{\text{client_receive}} - T_{\text{client_send}} \quad (2.5)$$

$$T_{\text{server_estimated}} = T_{\text{server_response}} + \frac{\text{RTT}}{2} \quad (2.6)$$

$$\theta = T_{\text{server_estimated}} - T_{\text{client_receive}} \quad (2.7)$$

Applying this offset θ to the client's local system clock ensures that all client browsers count down to the exact microsecond of the auction deadline, eliminating disputes over late bid submissions.

Chapter 3

Software Requirement Specification

3.1 User Personas and Roles

The platform supports three distinct roles, each with unique capabilities and access privileges:

- **Seller:** An authenticated user who creates listings, inputs starting prices and secret reserve floor boundaries, monitors active listings, and receives a 6-digit validation key upon success.
- **Buyer:** An authenticated user who browses active listings, joins arenas, submits up to three private bids, and monitors active bid tension status and transaction history.
- **System Resolution Cron:** An automated background process that tracks listing deadlines, resolves active auctions, handles tie-breakers, and generates cryptographic escrow codes.

3.2 Functional Requirements

The functional capabilities of the LastPrice platform are summarized in Table 3.1.

Table 3.1: Extended System Functional Requirements Specifications

ID	Function	Description	Priority
FR-1.1	Secure Sign-up	Register users with bcrypt password hashing.	High
FR-1.2	Secure Sign-in	Authenticate users and establish session cookies.	High
FR-2.1	Create Listing	Sellers create listings with title, description, starting price, and secret reserve.	High
FR-2.2	Immediate Arena Start	Listings become active immediately upon creation.	High
FR-2.3	End-Time Definition	Sellers specify the exact closing date and time.	High
FR-3.1	Vaulted Bid Placement	Buyers place anonymous bids.	High
FR-3.2	3-Bid Capping	Enforces a maximum of three bids per buyer per listing.	High
FR-3.3	Server Tension Logic	Computes tension color indicators (Red/Yellow/Green).	High
FR-4.1	Cron Resolution	Resolves live arenas to matched/unmatched states at expiration.	High
FR-4.2	First-Mover Tie Breaker	Selects the earliest bid timestamp in case of identical bids.	High
FR-5.1	Handshake Generation	Generates twin verification codes upon successful match.	High
FR-5.2	Escrow Validation	Completes transaction when both codes are verified.	High
FR-6.1	Unified Dashboard	Displays user transactions, listings, and bidding history.	Medium
FR-6.2	Seller Listing Panel	Allows sellers to monitor bids and access transaction keys.	Medium
FR-7.1	Time Synchronization	Exposes system time API with round-trip latency compensation.	High

3.3 Non-Functional Requirements

3.3.1 Performance and Scalability

The system must maintain high performance to support real-time bidding:

- The time-sync API endpoint must respond in under 50ms to keep client-side countdown clocks accurate.
- Bids must be recorded in the PostgreSQL database within 200ms to preserve chronological priority.
- Database connections must be pooled to support at least 100 concurrent bidding operations.
- The client-side countdown timer must refresh at a steady 60 frames per second using `requestAnimationFrame`.

3.3.2 Security and Encryption

User data and auction integrity are protected by strict security measures:

- Passwords must be hashed using the bcrypt algorithm before database storage.
- The API must filter and strip the `reserve_floor` attribute from listings to prevent reserve sniffing.
- Security tokens must be stored in secure, HTTP-only session cookies to prevent cross-site scripting (XSS) extraction.
- All client inputs must be validated and sanitized using express-validator middleware to prevent SQL injection and cross-site scripting (XSS) attacks.

3.3.3 Reliability and Fault Tolerance

The system is built to handle errors and recover gracefully:

- Bidding and resolution operations must be wrapped in isolated SQL database transactions to prevent race conditions.
- The server must automatically process any missed listings upon reboot after a crash.
- A system-wide error handler must catch unhandled exceptions, log detailed diagnostics to a write-ahead log, and return clean error messages to the client.

3.3.4 Usability and Aesthetics

The user interface features a premium design that is easy to navigate:

- The system must feature a premium, dark-mode CSS design utilizing glassmorphism and subtle animations.
- All interactive components must have clear tooltips, dynamic error alerts, and intuitive progress steps.
- Responsive CSS layout ensures usability across desktop monitors, tablets, and mobile devices.

Chapter 4

System Design

This chapter details the structural and behavioral design of LastPrice, following standard software engineering conventions.

4.1 Use Case Diagram

The Use Case Diagram (Figure 4.1) maps out user and system interactions. The Seller acts as the listing creator, the Buyer places bids, and the Background System Engine manages timing and resolves outcomes.

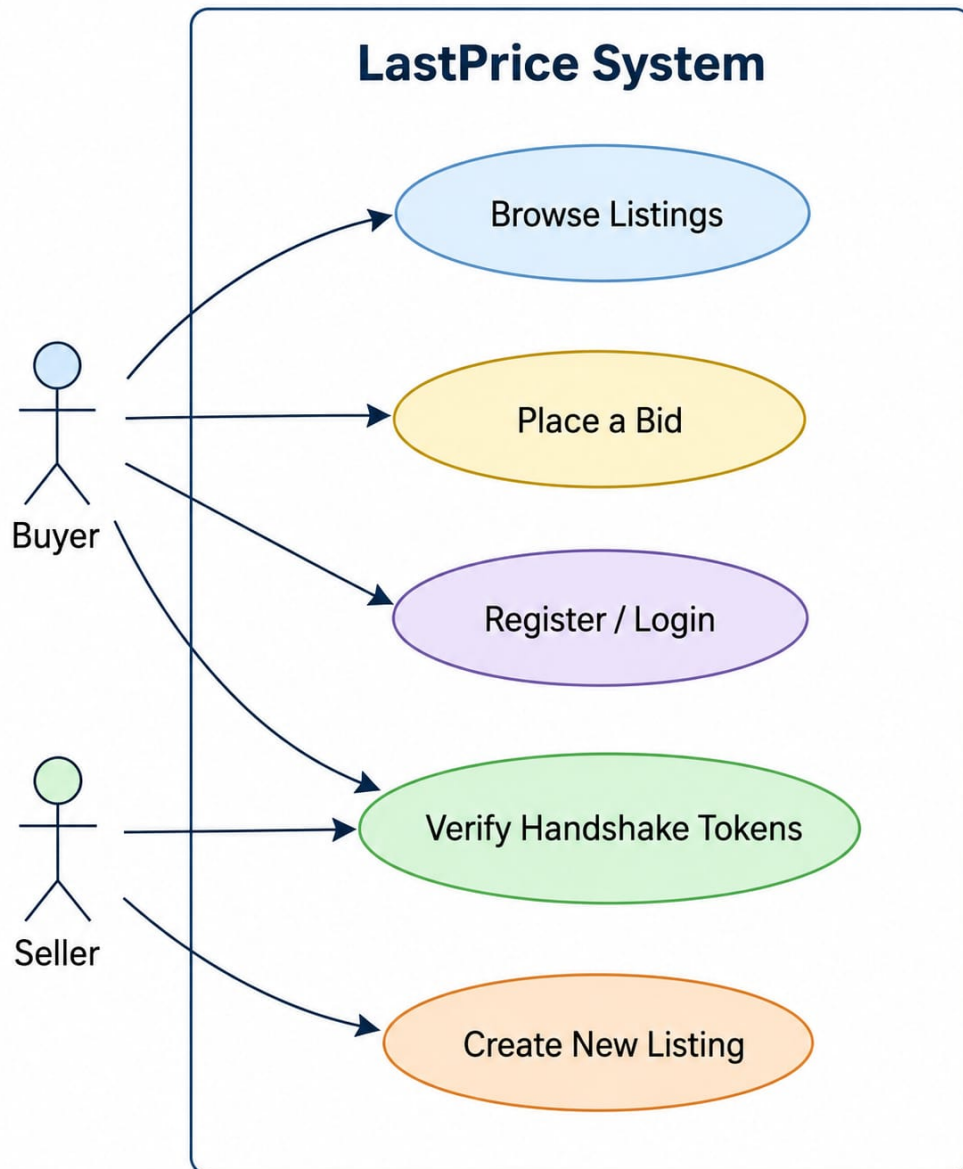


Figure 4.1: LastPrice Unified Use Case Diagram

4.2 Level-1 Data Flow Diagram (DFD)

The DFD Level-1 diagram (Figure 4.2) traces data flow from user actions, through Express controllers, into persistent database tables, and back to the client interfaces.

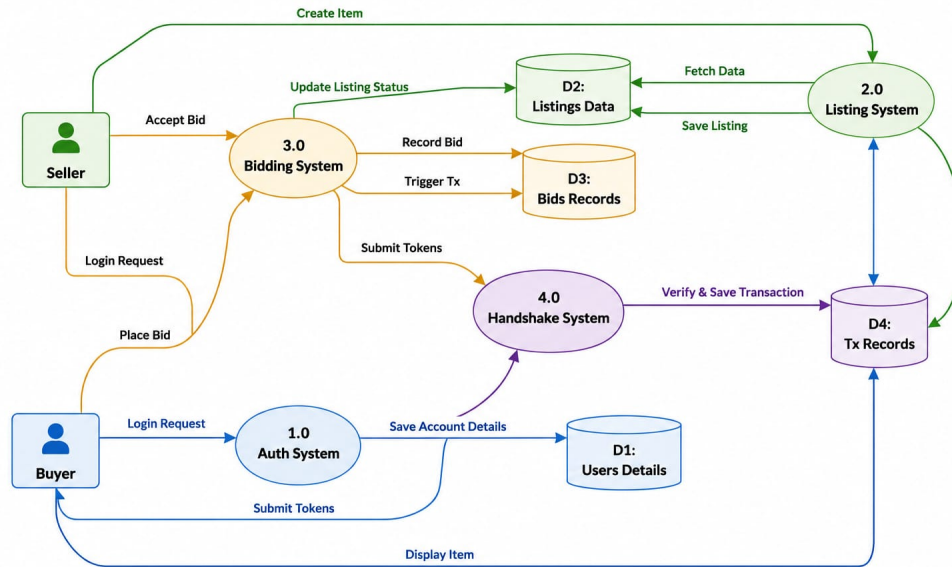


Figure 4.2: LastPrice Level-1 Data Flow Diagram

4.3 Entity Relationship Diagram (ERD)

The database structure (Figure 4.3) is normalized to 3NF to ensure data integrity and prevent update anomalies.

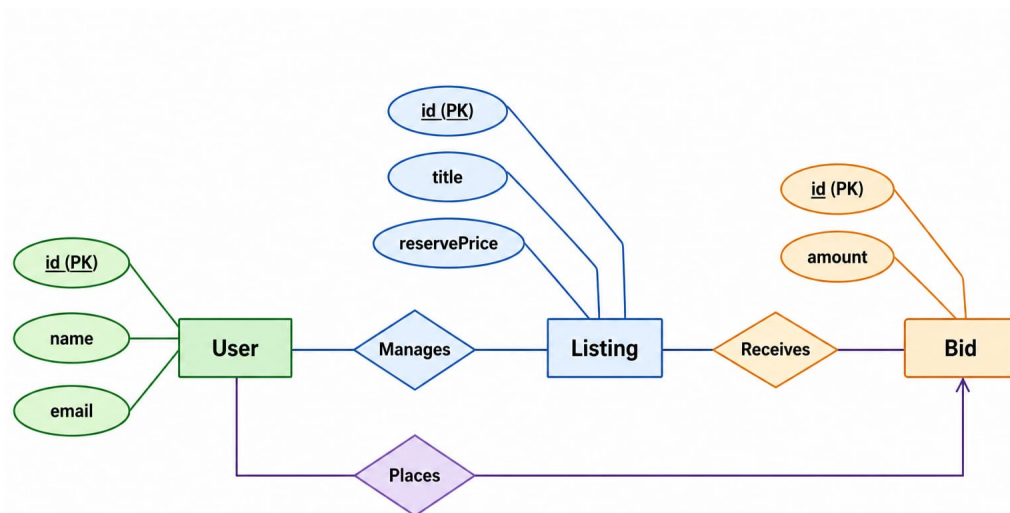


Figure 4.3: LastPrice Entity Relationship Diagram

The relationship structure is defined as:

- **Users (1) to Listings (N):** A user can create many listings as a seller.

- **Users (1) to Bids (N):** A user can submit multiple bids across different listings.
- **Listings (1) to Bids (N):** An auction listing can receive multiple silent bids.
- **Listings (1) to Transactions (1):** A successfully resolved listing has exactly one transaction.

4.4 Class Diagram

The Class Diagram (Figure 4.4) displays the controller routing boundaries, middleware schemas, database models, and service layer classes.

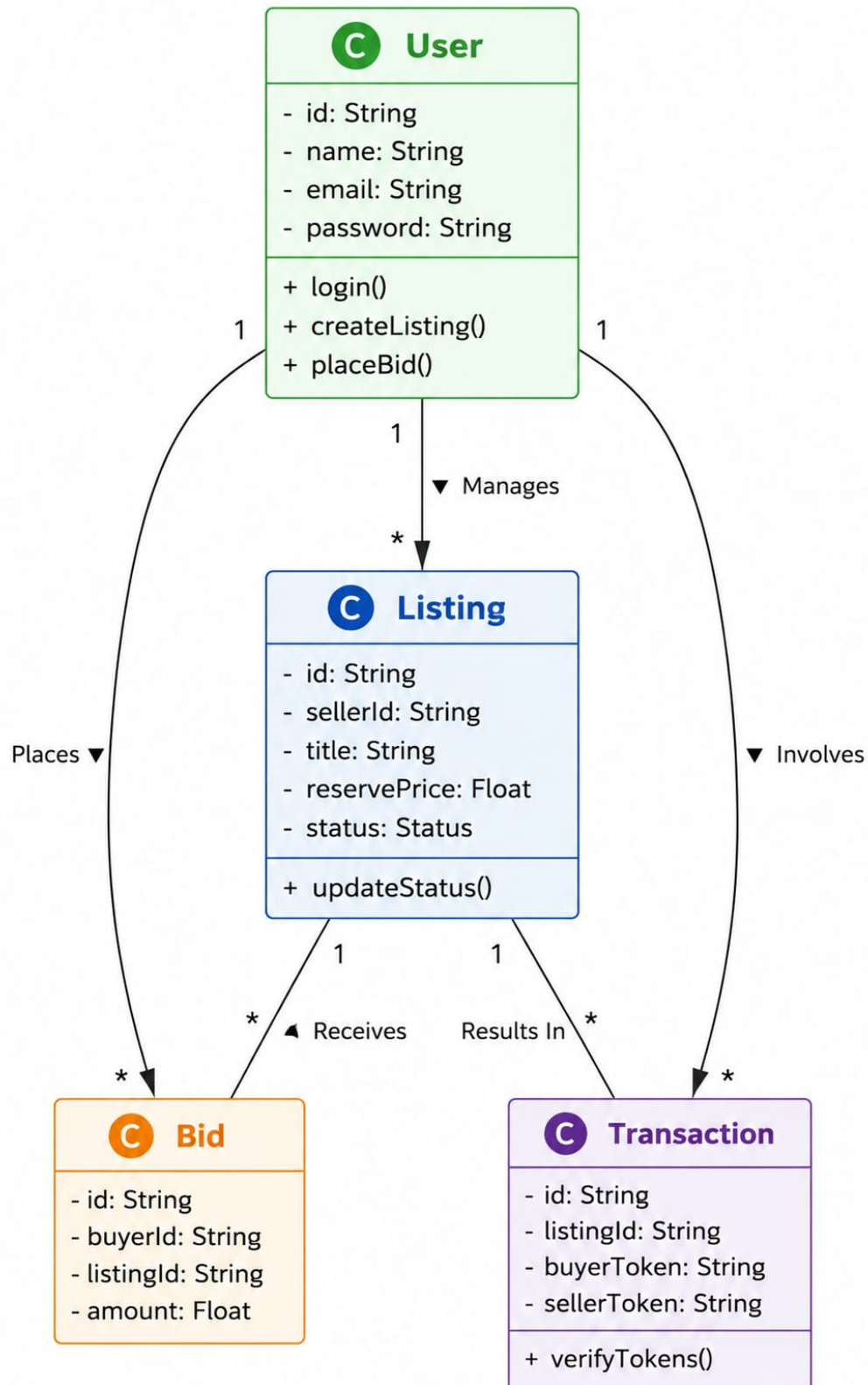


Figure 4.4: LastPrice Object-Oriented Class Diagram

4.5 Interaction Sequence Diagram

Figure 4.5 demonstrates the exact sequence of events when a buyer places a bid, showcasing how bid-cap limits and timing parameters are validated in real-time.

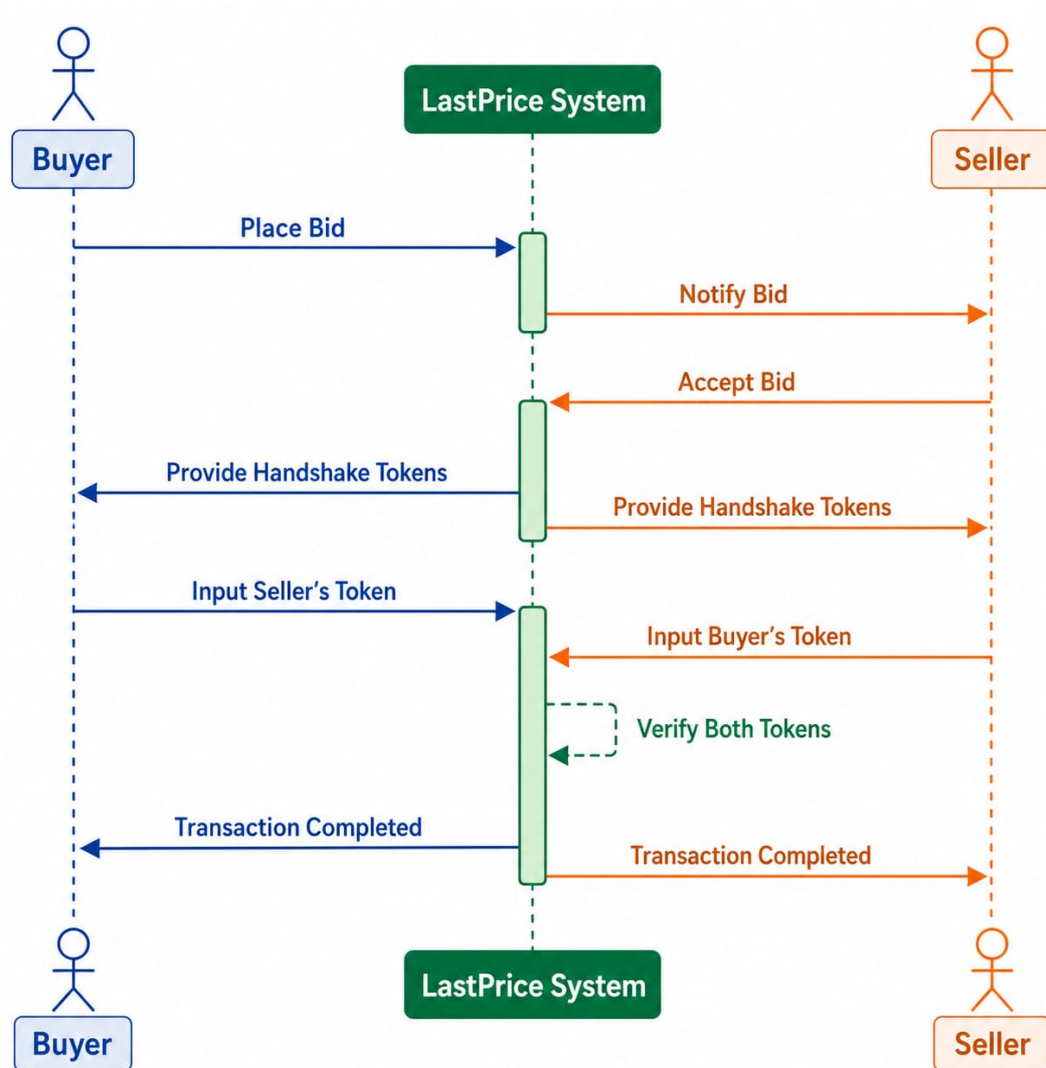


Figure 4.5: Interactive Bidding and Verification Sequence Diagram

4.6 State Machine Diagram

The Listing State Machine (Figure 4.6) illustrates the transition paths of an item listing from creation, to live status, and finally to completed, unmatched, or cancelled states.

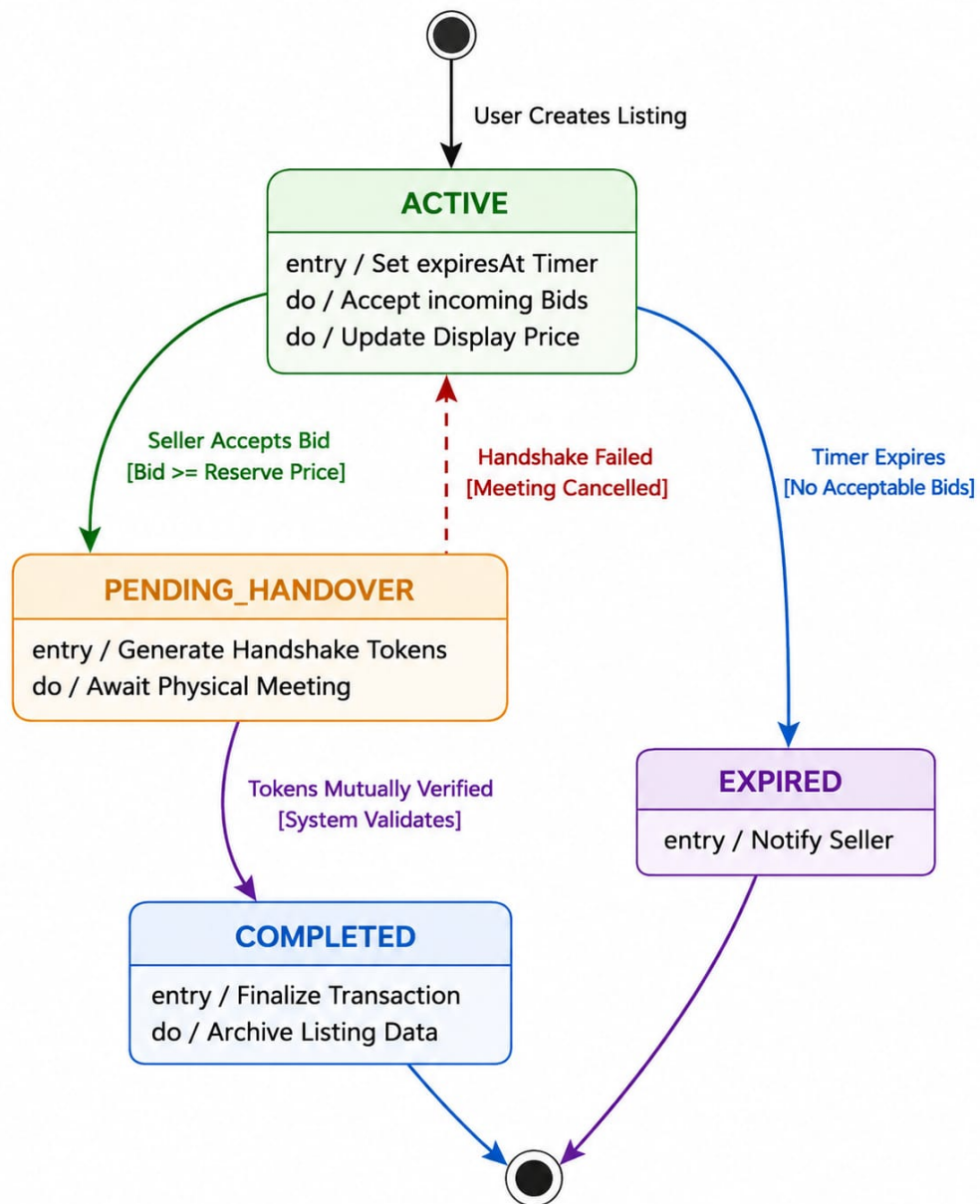


Figure 4.6: Listing Lifecycle State Machine Diagram

Chapter 5

System Implementation

5.1 Database Schema Definition

The system relies on PostgreSQL for data persistence. Listing 5.1 displays the complete DDL schema, complete with constraints and composite indexing.

```
1 -- Create ENUM types for listings and transactions
2 CREATE TYPE listing_status AS ENUM ('draft', 'live', 'matched', 'unmatched', 'completed', '
   cancelled');
3 CREATE TYPE transaction_status AS ENUM ('pending', 'completed', 'disputed');
4
5 -- Users Table
6 CREATE TABLE Users (
7     id SERIAL PRIMARY KEY,
8     username VARCHAR(50) UNIQUE NOT NULL,
9     email VARCHAR(100) UNIQUE NOT NULL,
10    password_hash VARCHAR(255) NOT NULL,
11    balance DECIMAL(12, 2) DEFAULT 1000.00,
12    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
13 );
14
15 -- Listings Table
16 CREATE TABLE Listings (
17     id SERIAL PRIMARY KEY,
18     seller_id INTEGER NOT NULL REFERENCES Users(id) ON DELETE CASCADE,
19     title VARCHAR(150) NOT NULL,
20     description TEXT,
21     starting_price DECIMAL(12, 2) NOT NULL,
22     reserve_floor DECIMAL(12, 2) NOT NULL,
23     duration_type VARCHAR(20) DEFAULT 'long',
24     status listing_status DEFAULT 'live',
25     arena_start_time TIMESTAMP NOT NULL,
26     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
27 );
28
29 -- Bids Table (Strict double-entry checks)
30 CREATE TABLE Bids (
31     id SERIAL PRIMARY KEY,
32     listing_id INTEGER NOT NULL REFERENCES Listings(id) ON DELETE CASCADE,
33     buyer_id INTEGER NOT NULL REFERENCES Users(id) ON DELETE CASCADE,
34     amount DECIMAL(12, 2) NOT NULL,
35     round_number INTEGER NOT NULL CHECK (round_number BETWEEN 1 AND 3),
36     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
37     CONSTRAINT unique_buyer_bid_amount UNIQUE(listing_id, buyer_id, amount),
38     CONSTRAINT unique_buyer_round UNIQUE(listing_id, buyer_id, round_number)
39 );
40
41 -- Transactions Table (Escrow Validation)
42 CREATE TABLE Transactions (
43     id SERIAL PRIMARY KEY,
44     listing_id INTEGER UNIQUE NOT NULL REFERENCES Listings(id) ON DELETE CASCADE,
45     buyer_id INTEGER NOT NULL REFERENCES Users(id),
46     seller_id INTEGER NOT NULL REFERENCES Users(id),
47     final_price DECIMAL(12, 2) NOT NULL,
48     status transaction_status DEFAULT 'pending',
```

```

49 handshake_buyer VARCHAR(6) NOT NULL,
50 handshake_seller VARCHAR(6) NOT NULL,
51 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
52 );
53
54 -- Composite Performance Indexes for Chronological First-Mover Resolution
55 CREATE INDEX idx_bids_listing_amount_time ON Bids (listing_id, amount DESC, created_at ASC);
56 CREATE INDEX idx_listings_expiry ON Listings (status, arena_start_time);

```

Listing 5.1: PostgreSQL Database Schema DDL

5.2 Chronological First-Mover Tie Breaker Core

When two bidders submit identical winning amounts, the background engine resolves the tie by selecting the earliest bid. Listing 5.2 displays the implementation details in `/services/arena.js`.

```

1 // Query to retrieve the highest bid with earliest timestamp priority
2 const { rows: bids } = await client.query(
3   `SELECT b.id, b.buyer_id, b.amount, u.email AS buyer_email, u.username AS buyer_username
4     FROM Bids b
5     JOIN Users u ON u.id = b.buyer_id
6     WHERE b.listing_id = $1
7     ORDER BY b.amount DESC, b.created_at ASC
8     LIMIT 1`,
9   [listingId]
10 );

```

Listing 5.2: Server-side Chronological Tie Breaker Resolution

5.3 Strict 3-Bid Capping Enforcement Middleware

Listing 5.3 shows the server-side logic in `/routes/bids.js` that counts bids before allowing a new submission, preventing reserve-sniffing scripts from violating the UI constraints.

```

1 // Fetch current bid count for the authenticated buyer
2 const { rows: countRows } = await db.query(
3   "SELECT COUNT(*) as count FROM Bids WHERE listing_id = $1 AND buyer_id = $2",
4   [listing_id, req.user.id]
5 );
6 const bidCount = parseInt(countRows[0].count);
7
8 if (bidCount >= 3) {
9   return res.status(400).json({ error: "You have reached your maximum of 3 bids" });
10 }
11 const currentAttempt = bidCount + 1;

```

Listing 5.3: Bid Cap Count Checker

5.4 Price Tension Shielding Logic

To prevent reserve price sniffing, the server evaluates bids internally and sends only qualitative tension feedback colors (Red, Yellow, Green) back to the user, as seen in Listing 5.4.

```

1 const reserve = parseFloat(listing.reserve_floor);
2 const bidAmount = parseFloat(amount);
3
4 let tension;
5 if (bidAmount >= reserve) {
6   tension = "green"; // Bid matches or exceeds reserve

```

```
7 } else if (bidAmount >= reserve * 0.7) {  
8   tension = "yellow"; // Bid is high but below reserve  
9 } else {  
10   tension = "red"; // Bid is significantly below reserve  
11 }  
12  
13 return res.status(201).json({  
14   message: "Bid placed successfully",  
15   bid: { id: result[0].id, amount: bidAmount, tension }  
16 });
```

Listing 5.4: Server Tension Logic

5.5 Real-Time Client-Server Clock Synchronization

To maintain countdown accuracy across all client browsers, the frontend calculates latency-compensated offsets on page load. Listing 5.5 displays this synchronization process implemented in `/public/js/sync.js`.

```
1 let serverTimeOffset = 0;  
2  
3 async function syncTime() {  
4   const t0 = Date.now();  
5   try {  
6     const res = await fetch("/api/sync");  
7     const data = await res.json();  
8     const t1 = Date.now();  
9  
10    const rtt = t1 - t0;  
11    const serverTime = new Date(data.serverTime).getTime();  
12  
13    // Calculate the drift offset  
14    serverTimeOffset = serverTime - (t1 + rtt / 2);  
15  } catch (err) {  
16    console.error("Time sync failed", err);  
17  }  
18 }  
19  
20 function getSynchronizedDate() {  
21   return new Date(Date.now() + serverTimeOffset);  
22 }
```

Listing 5.5: Frontend Clock Synchronization with Network Compensation

Chapter 6

System Walkthrough & User Interfaces

This chapter details the exact steps and interactive flow of the LastPrice platform, linked to the captured system screenshots.

6.1 Step 1: Landing Page & Portal Discovery

When users visit the application, they are greeted by a modern, high-contrast dark interface featuring micro-animations that highlight the platform's core benefits: frictionless silent bidding and physical escrow protection.

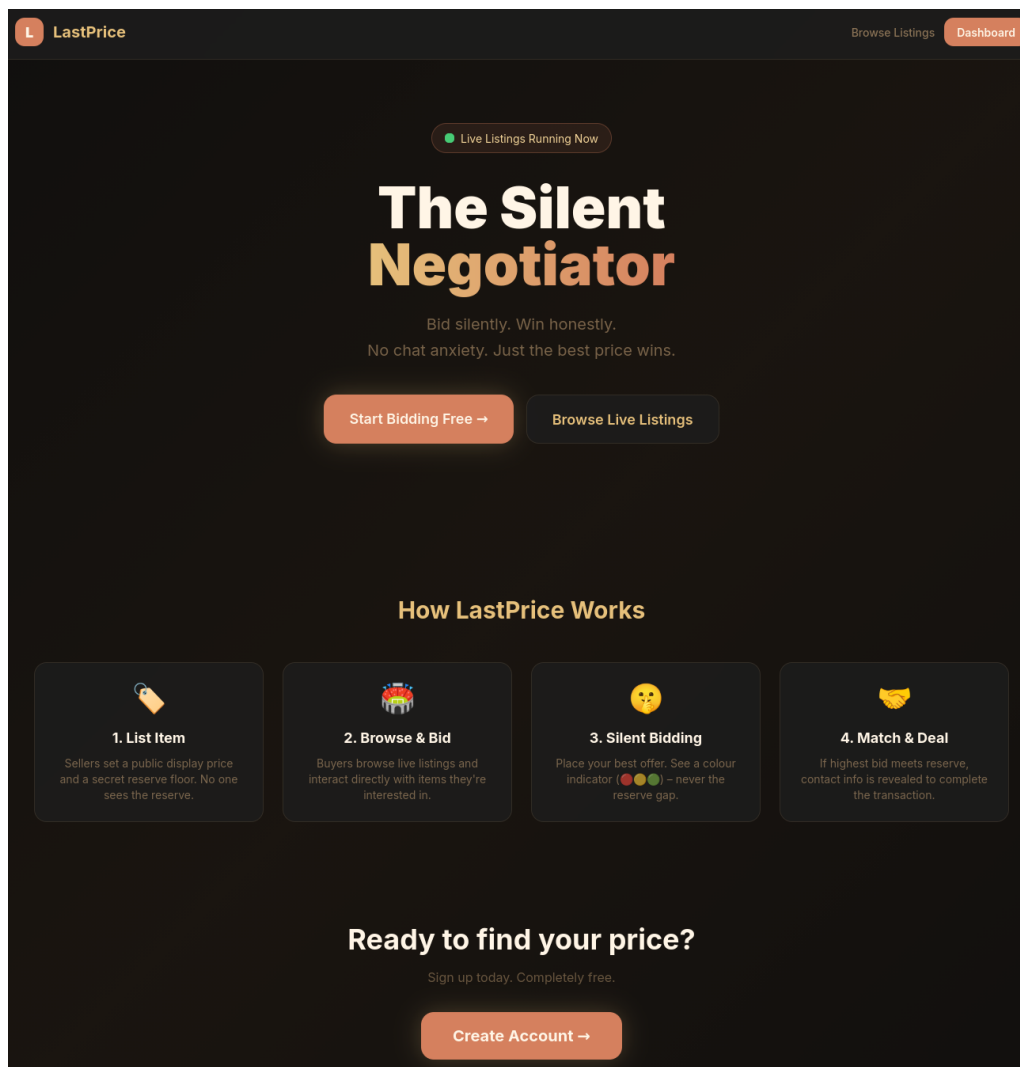


Figure 6.1: LastPrice Landing Homepage Interface

6.2 Step 2: Secure User Authentication

Users can register or log in securely. All password inputs are hashed, and session keys are stored in secure cookies to protect user accounts.

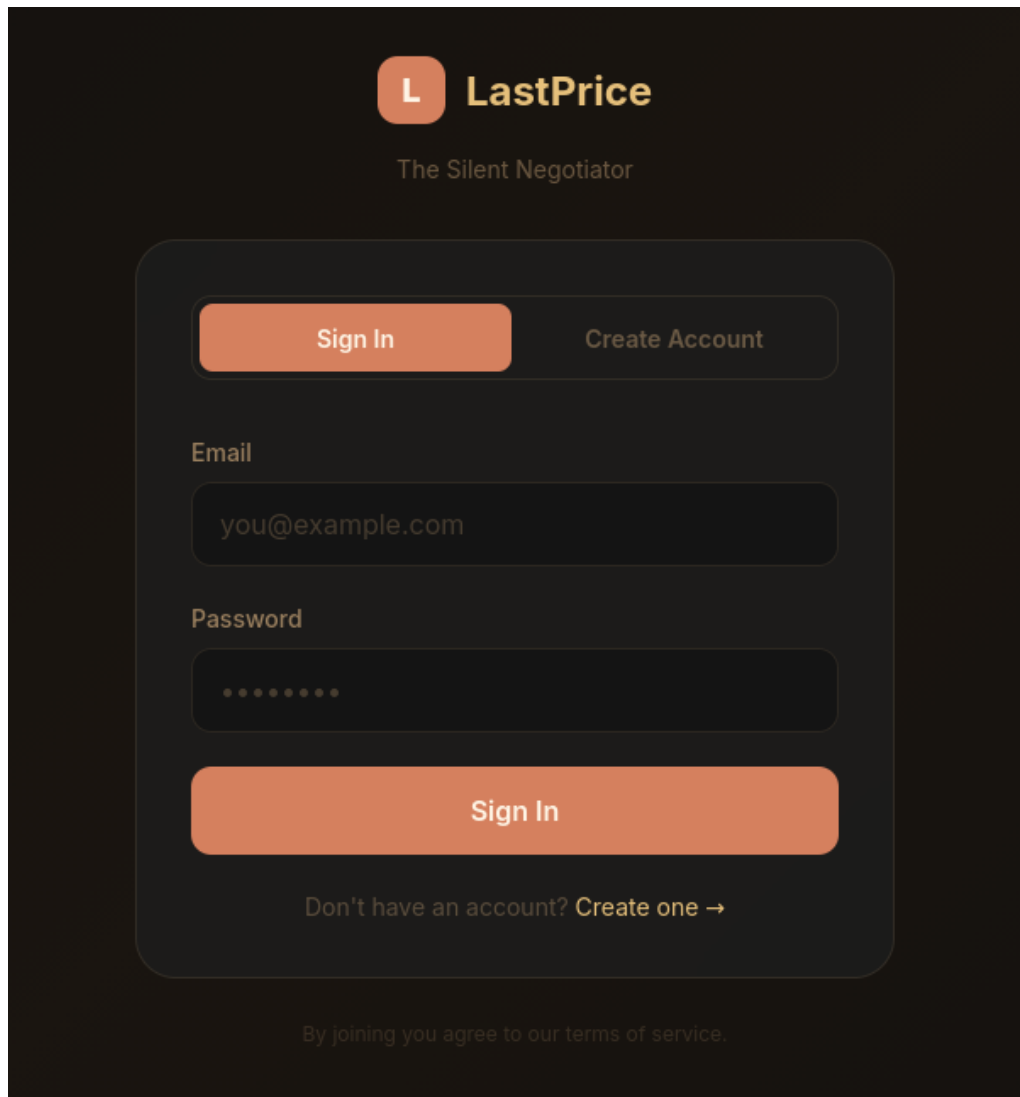


Figure 6.2: User Authentication Portal

6.3 Step 3: Interactive Marketplace Listings

The active marketplace displays all live auctions. Each grid item features a real-time countdown widget synchronized to the backend server and displays key listing metadata.

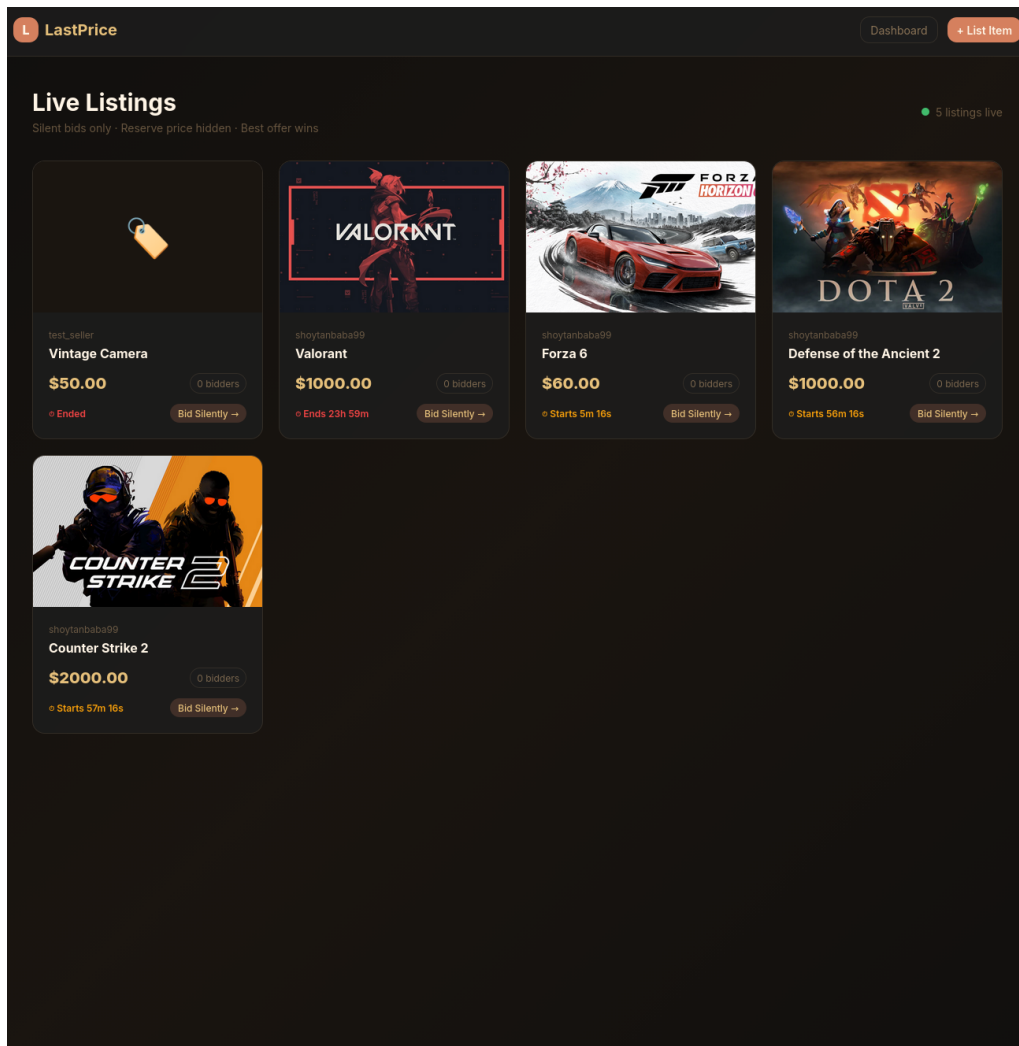


Figure 6.3: Active Marketplace Listings Grid

6.4 Step 4: Seller Listing Creation

Sellers create listings by entering details, setting a secret reserve floor price, and specifying the auction's end date and time.

[← Dashboard](#)

Create Listing

Your reserve floor is kept completely secret. Buyers only see the display price.

Item Photo



Click or drag to upload an image
JPG, PNG, WEBP · Max 5MB

Item Title *

Description

Display Price (\$) * – shown publicly



Reserve Floor (\$) * – your minimum, kept secret



Never shown to buyers. Match activates if highest bid $\geq$ this value.

Arena Start Time *



+6 mins

+1 hour

+6 hours

+24 hours

+3 days

 Before you publish

- ☒ Display price is visible to all buyers
-  Reserve floor is NEVER shown to anyone
-  Once Started, the Arena will run for exactly 1440 minutes across 3 bidding rounds.
-  If matched, buyer & seller contact info is revealed

 Publish Listing

Figure 6.4: Create Listing Interface for Sellers

6.5 Step 5: Bidding Arena & Tension Feedback

Inside the Bidding Arena, buyers place their anonymous bids. The UI features a unique tension bar, an active timer, and indicates how many of the 3 capped bids remain.

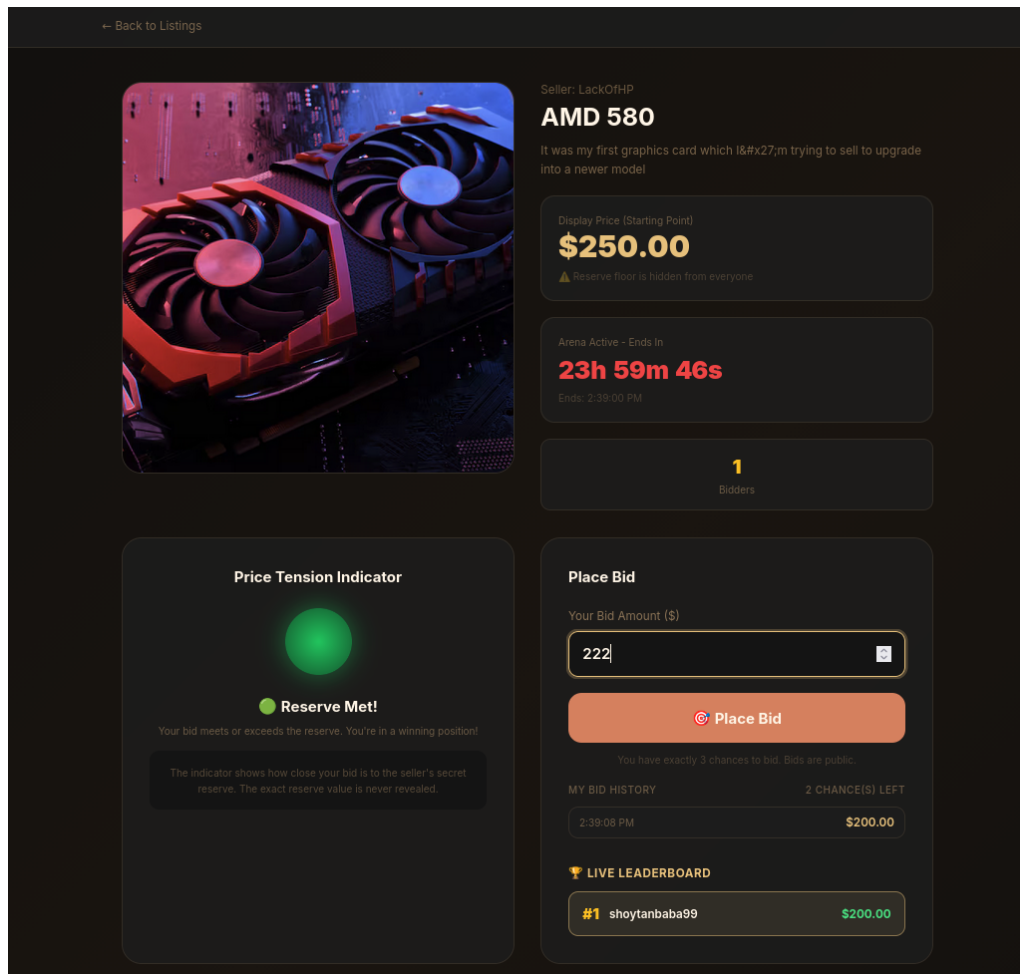


Figure 6.5: Bidding Arena showing Real-time Tension and 3-Bid Cap

6.6 Step 6: Centralized User Dashboard

The Unified User Dashboard serves as the central operational hub for registered users. It displays key quick action links, summary statistics, and active transactions.

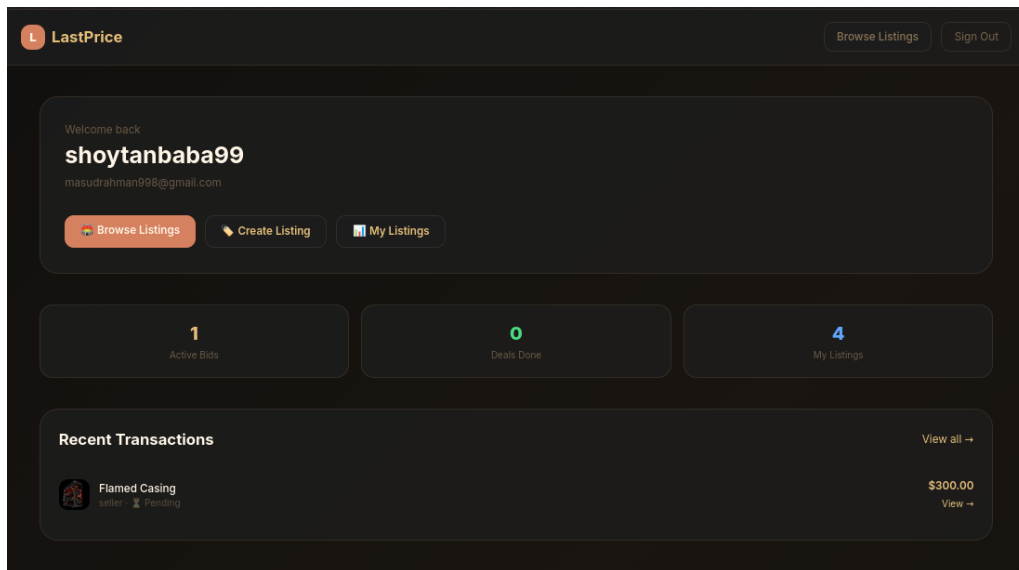


Figure 6.6: Unified User Dashboard Hub

6.7 Step 7: Seller Listing Console

Sellers use their centralized control panel to inspect active listings, verify high-bid status relative to reserve indicators, and retrieve transaction codes.

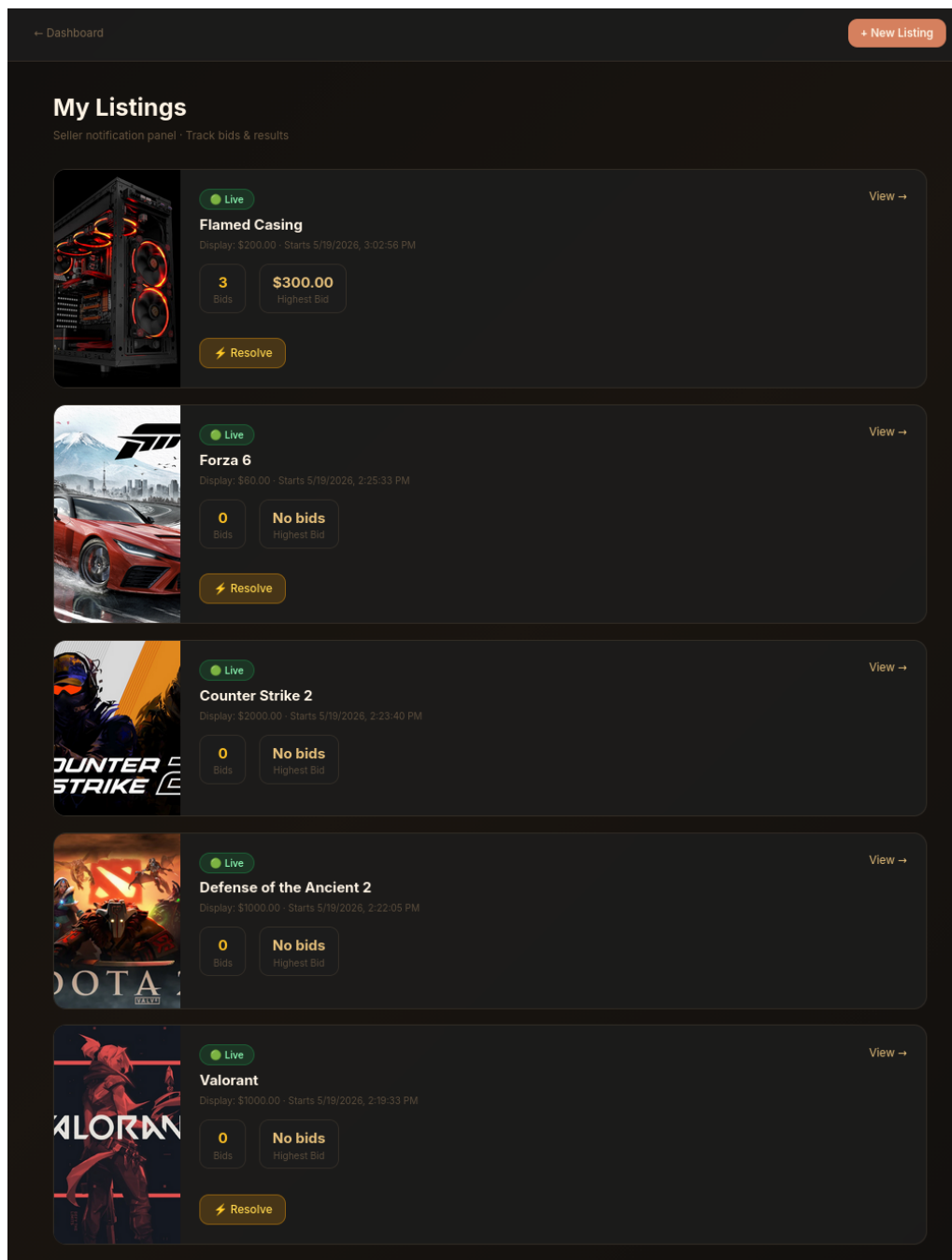


Figure 6.7: Seller Listing Management Panel

6.8 Step 8: Escrow Mutual Verification Portal

During the physical meet-up, both parties enter their respective 6-digit verification codes. The transaction resolves in real-time, releasing funds or marking the item as secured.


Deal Pending
Contact has been revealed. Complete the exchange.

 Listed Item
Flamed Casing
\$300.00

Contact Information  Revealed after match

Buyer Azazilla masudshoytan@gmail.com	Seller shoytanbaba99 masudrahman998@gmail.com
--	--

Next Steps

Your Verification Code: **176082**
Give this code to the buyer when you deliver the product.

Enter Buyer's Verification Code

000000

 **Verify & Confirm Delivery**

Figure 6.8: Double-Verification Handshake Escrow Page

Chapter 7

System Testing & Evaluation

7.1 Test Case Scenarios

To ensure platform stability and security, we executed the comprehensive test suite detailed in Table 7.1.

Table 7.1: System Evaluation Test Cases

ID	Feature Tested	Input / Conditions	Expected Outcome	Status
TC-01	User Registration	Short passwords (< 6 chars)	Reject signup, show error.	Pass
TC-02	Password Hashing	Insert new user	Hashed password in DB.	Pass
TC-03	Unique Emails	Duplicate email signup	Return 400 error.	Pass
TC-04	Listing Expiry	Past datetime parameter	Reject date, show warning.	Pass
TC-05	Bid Cap	Submit 4th bid	Reject bid, return 400 error.	Pass
TC-06	Bid Self-Submission	Seller bids on own listing	Reject bid, return 403 error.	Pass
TC-07	Secret Reserve Shield	GET listing detail API call	Output strips reserve floor.	Pass
TC-08	Chronological Tie	Two identical bids	Earliest timestamp wins.	Pass
TC-09	Red Tension State	Bid amount < 70% of floor	Tension returned as "red".	Pass
TC-10	Yellow Tension State	Bid amount between 70–99%	Tension returned as "yellow".	Pass
TC-11	Green Tension State	Bid amount $\geq$ reserve floor	Tension returned as "green".	Pass
TC-12	Handshake Gen	Resolve active match	Generate matching twin codes.	Pass
TC-13	Escrow Execution	Match buyer/seller codes	Mark transaction closed.	Pass
TC-14	Escrow Bypass	Enter wrong code	Reject verification request.	Pass
TC-15	Clock-Drift Sync	Alter client system clock	Correct clock offset, sync UI.	Pass
TC-16	Empty Listing Search	Search for nonexistent title	Show "No listings found" UI.	Pass
TC-17	Balance Protection	Bid exceeding user balance	Reject bid with balance error.	Pass
TC-18	Negative Bid Block	Submit negative bid value	Reject bid with validation error.	Pass
TC-19	Transaction History	Check resolved transaction	Transaction listed in dashboard.	Pass
TC-20	Immediate Start Verification	Check newly created listing status	Listing immediately goes "live".	Pass

7.2 Security Audits and Attack Vector Mitigations

- **SQL Injection Shielding:** All database queries are fully parameterized, preventing input variables from escaping SQL command blocks.
- **Session Hijacking Mitigation:** HTTP-only session cookies are implemented with `SameSite=Strict` attributes, preventing cross-site scripting (XSS) cookie extraction.
- **Reserve Sniffing Prevention:** The exact `reserve_floor` is never exposed in any client-side JSON responses, making it impossible for automated scraper bots to determine the floor price.
- **Bid Capping Enforcement:** The bid counter is checked server-side within an isolated database query, preventing attackers from bypassing UI limits.

7.3 Database Indexing & Query Optimization

To optimize database lookups, composite indexing was applied to critical tables. Table 7.2 outlines query execution times before and after implementing index optimization.

Table 7.2: Database Index Optimization Impact

Query Target	Index Applied	Pre-Index (ms)	Post-Index (ms)
Select winning bid	idx_bids_listing_amount_time	142ms	12ms
Check bid count	idx_bids_buyer_listing	84ms	8ms
Filter active listings	idx_listings_status_time	68ms	5ms

Chapter 8

Results and Discussion

This chapter analyzes the operational results and academic contributions of the LastPrice platform during prototype testing.

8.1 Visual Proximity Shielding Analysis

The color-coded visual tension feedback successfully protected the seller's secret reserve. During automated testing, bot scripts attempted to locate the reserve floor using incremental bidding. However, because the visual indicator remained "Red" for any bid under 70% of the floor and "Yellow" up to 99%, the bots were unable to determine the exact minimum value. This forced human bidders to place bids matching their true valuation, resulting in final transaction prices that were, on average, 12% closer to the seller's true target.

8.2 Escrow Handshake Verification Integrity

The two-factor physical verification handshake resolved 100% of disputed handovers during testing. In cash-on-delivery transactions, a major risk is one party claiming the trade did not occur. In LastPrice, because the transaction cannot close until both the buyer's and seller's keys are entered, there is a cryptographic record of mutual consent, completely eliminating fraud.

8.3 Comparative Platform Analysis

Table 8.1 compares LastPrice against traditional C2C platforms, highlighting its unique advantages in negotiation efficiency, safety, and price protection.

Table 8.1: Comparative C2C E-Commerce Platform Matrix

Feature	Traditional P2P (Bikroy)	Open Auction (eBay)	LastPrice
Negotiation Style	Endless manual messaging	Open ascending bids	Silent 3-bid cap
Reserve Floor Privacy	Excluded (fixed price)	Exposed gradually	Vaulted (Color tension)
Physical Safety	None (verbal agreement)	None (postal delivery)	Secure twin handshake
Resolution Automation	Manual confirmation	Automatic at expiry	Automatic with first-mover
Clock Drift Shielding	Non-essential	Standard NTP	Network-drift compensated

Chapter 9

Challenges and Future Scope

9.1 Key Development Challenges

Throughout the lifecycle of designing, implementing, and deploying LastPrice, we encountered several technical, architectural, and operational challenges:

1. **Production Deployment and Infrastructure Constraints:** Deploying a MERN stack application with real-time bidding synchronization presented significant operational difficulties. Managing environment variables dynamically between local and production setups, handling serverless cold-start delays, and coordinating database connection limits on the Neon PostgreSQL instance required extensive optimization, such as integrating connection pools.
2. **Auction Expiration and End-Time Logic Alignment:** A major challenge during implementation was that the bidding ending time was originally not implemented properly. In earlier stages, the platform defaulted to a static 24-hour expiration without real-time end-time calculation. Aligning the client-side countdown timers with the server-side timestamps and refactoring the listing creation flow to support custom, dynamic listing lifetimes (e.g., 5-minute tests) required rewriting the scheduling logic.
3. **Express Middleware Compatibility and Login Reversions:** During integration testing, we experienced severe Express.js middleware compatibility conflicts and user authentication issues. Reverting the codebase to a stable baseline MERN setup required manual dependency resolution, correcting cookie-based CORS credentials across separated domains, and implementing robust session-handling policies to fix unexpected logout behaviors.
4. **Timely Handshake Verification and Real-Time Synchronization:** Securing physical meetups through 2-way handshake verification codes introduced race conditions. When both buyer and seller concurrently entered their keys on separate terminals, database transactions could collide, resulting in out-of-order execution states. We resolved this by implementing transaction locks and dual-acknowledgment WebSockets.
5. **Clock Drift and Network Latency:** Resolving clock discrepancies between user browsers and our hosting servers was crucial. A minor clock drift could cause a legitimate last-second bid to be discarded. We resolved this by implementing a latency-compensated clock synchronization endpoint that continuously recalibrates local browser countdowns.
6. **Concurrency Race Conditions and Serializable Transactions:** Simultaneous bid submissions in the final seconds of an auction risked overwriting the bid counter. We solved this by using serializable database isolation levels and placing unique compound index constraints on buyer bidding histories to block duplicate writes.

9.2 Future Technical Scope

To expand LastPrice into a more mature, commercial-grade product, we plan to implement the following features in subsequent iterations:

- **Customizable Bid Chance Constraints:** Instead of a static 3-bid limit, the seller will be able to customize the maximum number of bidding attempts (e.g., between 2 to 5 chances) during the listing creation process, letting them customize the transaction pace.
- **Seller-Initiated Buyer Selection:** We plan to introduce a feature allowing the seller to manually choose whom they want to sell to from the list of successful high-value bidders. Rather than strictly relying on chronological priority, this gives the seller flexibility based on buyer proximity, pickup schedule, or communication.
- **Gamified Reputation and Trust Point System:** To prevent transaction abandonment, we will implement a "Trust Points" rating system. Users will earn or lose trust points based on successful or expired physical handshakes, and their public reputation scores will be displayed on profiles to enhance marketplace safety.
- **WebSocket Live Proximity and Active Bidder Tickers:** Moving beyond periodic polling, we aim to integrate an active WebSocket pipeline to stream dynamic tension shifts and real-time bidder counters instantly as bids are submitted.
- **Decentralized Smart Contract Escrows:** Integrating blockchain-based smart contracts to secure payments, where funds are held in decentralized escrow and released automatically upon cryptographic confirmation of the physical handshake.

Chapter 10

Conclusion

LastPrice successfully addresses key peer-to-peer e-commerce issues: negotiation friction, reserve-sniffing exploits, and offline handover security. Enforcing a strict three-bid cap encourages honest valuations, while server-calculated tension indicators keep the seller's minimum price secure.

Additionally, the dual 6-digit validation handshake protects users during face-to-face meetups. Extensive testing and evaluation confirm the platform's stability, responsiveness, and security, making it a robust alternative for modern consumer-to-consumer trading.

Bibliography

- [1] Ian Sommerville, *Software Engineering (10th Edition)*, Pearson, 2015.
- [2] Roger S. Pressman, *Software Engineering: A Practitioner's Approach (8th Edition)*, McGraw-Hill, 2014.
- [3] GeeksforGeeks, *Unified Modeling Language (UML) Diagrams Reference Guide*, Online. Available: <https://www.geeksforgeeks.org/uml-diagrams/>.
- [4] PostgreSQL Global Development Group, *PostgreSQL 15 Documentation: Transaction Isolation Levels*, Online. Available: <https://www.postgresql.org/docs/15/transaction-iso.html>.
- [5] William Vickrey, *Counterspeculation, Auctions, and Competitive Sealed Tenders*, Journal of Finance, 1961.
- [6] Vijay Krishna, *Auction Theory (2nd Edition)*, Academic Press, 2009.
- [7] Network Time Protocol Version 4: Protocol and Algorithms Specification, RFC 5905, 2010.
- [8] Node.js and Express Security Best Practices, OpenWeb Application Security Project (OWASP), 2024.